

std::move

- ДАВАЙТЕ РАЗБЕРЁМ НА ПРИМЕРЕ

```
int a = 5, b = 7;  
b = std::move(a);
```

a	b
5	7
?	5

НЕОПРЕДЕЛЁННОЕ,
НО КОРРЕКТНОЕ
СОСТОЯНИЕ

- lvalue → `std::move()` → rvalue

- Сами объекты в памяти физически НЕ "ПЕРЕЗЖАЮТ"

- Можно реализовать swap

```
void swap(int& a, int& b) {  
    int temp = std::move(a);  
    a = std::move(b);  
    b = std::move(temp);  
}
```

← ЭТО МАКСИМАЛЬНО
УЛУЧШЕННЫЙ SWAP



ОПЯТЬ КЛАССЫ: Move-конструктор

- `class-name(class-name&&);`
м.б. const
-//- = default; - by компилятор
-//- = delete; - ЕГО НЕТ

- ВАЖНОЕ ДОПОЛНЕНИЕ для move-и copy-конструкторов:

В ПАРАМЕТРАХ МОГУТ БЫТЬ ЕЩЁ ЗНАЧЕНИЯ ПО УМОЛЧАНИЮ

```
class-name(const class-name& a, int b = 5);
```

- Когда?

когда инициализируем с помощью rvalue

- ✓ ЯВНАЯ ИНИЦИАЛИЗАЦИЯ

```
my-class b;  
my-class a = std::move(b);  
my-class c(std::move(b));
```

- ✓ ПЕРЕДАЧА АРГУМЕНТА
В ФУНКЦИЮ

```
void f(my-class a);  
f(std::move(a));
```

- ✓ ВОЗВРАТ ЗНАЧЕНИЯ ИЗ
ФУНКЦИИ

```
my-class f(params) {  
    my-class a; // my-class есть  
    return a; // move-конструктор  
}
```

• НЕЯВНЫЙ move-конструктор

- Создается компилятором, мы НЕ ОПРЕДЕЛИЛИ:

1. move-конструктор (ЕГО самого)
2. деструктор
3. copy-конструктор
4. копирующие операции присваивания
5. move-операции присваивания

- ДЕЛАЕТ ПОДГОТОВКУ КОПИЮ ИЛИ `std::move` КАЖДОГО ПОЛЯ В ПОРЯДКЕ ИХ ОБЪЯВЛЕНИЯ В КЛАССЕ

• Move-ctor / Copy-ctor. КТО БУДЕТ ВЫЗВАН?

- ЕСТЬ ОБА (ЯВНЫЕ ИЛИ НЕЯВНЫЕ)

- lvalue $\rightarrow$ copy-ctor

- rvalue $\rightarrow$ move-ctor

- НЕТ move-ctor (`=delete`) $\rightarrow$ copy-ctor

Operator = (для lvalue)

• COPY-ПЕРЕГРУЗКА

```
my_class& my_class::operator=(my_class other) {
```

```
    this  $\rightarrow$  a = other.a
```

```
    ..
```

```
    return *this;
```

```
}
```

Operator = (для rvalue)

• COPY-ПЕРЕГРУЗКА

```
my_class& my_class::operator=(const my_class& other);
```

• MOVE-ПЕРЕГРУЗКА

```
my_class& my_class::operator=(my_class&& other);
```

⚠ ПРОВЕРЯЙТЕ, ЧТО НЕ ВЫЗВАЛИ САМИ ДЛЯ СЕБЯ (this и other),

ЧТОБЫ НЕ ОЧИСТИТЬ СЕБЯ (МОЖНО ИСПОЛЬЗОВАТЬ `swap`)

⚠ НЕ БРОСАЕТ ИСХОДИЩИЕ

Copy & Swap Idiom (ИДЕАЛЬНОЕ РЕШЕНИЕ)

```
my_class& operator=(my_class other) {
```

```
    swap(*this, other);
```

```
    return *this;
```

```
}
```

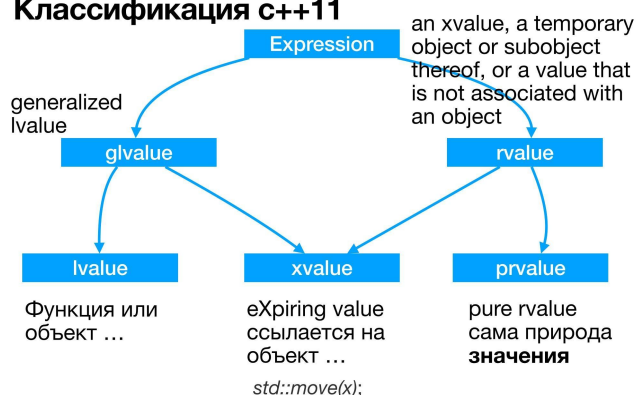


Доп классификация (вроде не так важна)

Классификация c++11



Классификация c++11



Правило 0/3/5

• Правило 3

Если есть один из них:

- деструктор
- конструктор
- операция присваивания

Скорее всего нужно будет определить все 3

• Правило 3, 5

То же самое + swap для copy&swap idiom

• Правило 5

Если нужна move-оптимизация (Пр.3 - блокирует создание неявного move-ctor)

Пр.3

⊕

- move-конструктор
- move-операция присваивания

• Правило 0

Если не нужна "тонкая" работа с памятью

не должно быть ничего из ctor, dtor и oper=

ООП. Возвращение

3) Наследование

Названия + общий язык

А БАЗОВЫЙ КЛАСС - Base
• РОДИТЕЛЬСКИЙ
• СУПЕРКЛАСС
↑
ВЫВЕДЕН ИЗ

Б Производный класс - Derived
• ДОЧЕРНИЙ
• ПОДКЛАСС

ВЕРНЁМСЯ К ТОРТИКАМ

- БАЗОВЫЙ КЛАСС: ТОВАР

```
class Product {  
    public:  
        void set-price(int p);  
        int get-price();  
    private:  
        int price;  
};
```

- НАСЛЕДУЕТСЯ (EgA)

```
class Food: public Product {  
    public:  
        void set-weight(int w);  
        int get-weight();  
    private:  
        int weight();  
};
```

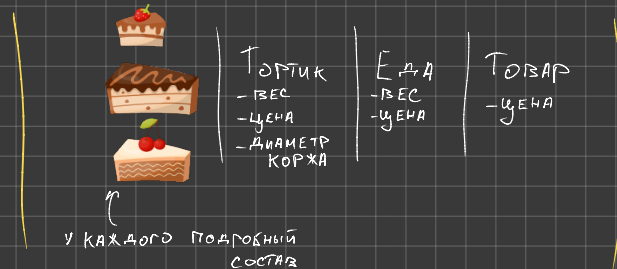
Food:

Product weight

↑
ЧЛЕНЫ КЛАССА Product

ЧЛЕНЫ
КЛАССА Food

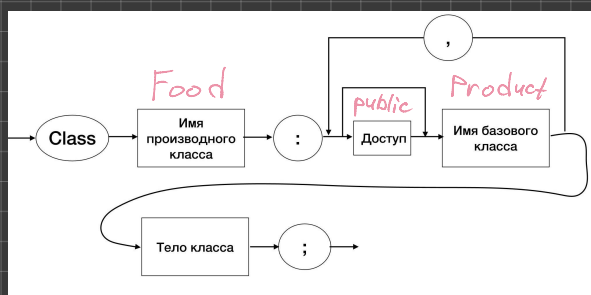
БАЗОВЫЙ КЛАСС НАСЛЕДУЕТСЯ
ПРОИЗВОДНЫМ КЛАССОМ



Product

↑
Food

ОБЩИЙ СИНТАКСИС НАСЛЕДОВАНИЯ



Спецификаторы доступа наследования

• public

БАЗОВЫЙ

public

protected

private

⇒

Производный

public

protected

X

• protected

БАЗОВЫЙ

public

protected

private

⇒

Производный

protected

protected

X

• private

БАЗОВЫЙ

public

protected

private

⇒

Производный

private

private

X



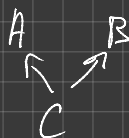
СПАТЬ НЕМНОГО ХОЧЕТСЯ...

Граф наследования

```
class A {}
```

```
class B {}
```

```
class C: public A, public B {}
```



Будем писать кратко:

```
A;
```

```
B: A;
```

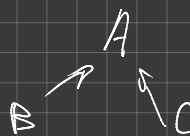
```
C: B;
```



```
A;
```

```
B: A;
```

```
C: A;
```



```
A;
```

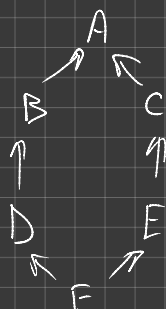
```
B: A;
```

```
C: A;
```

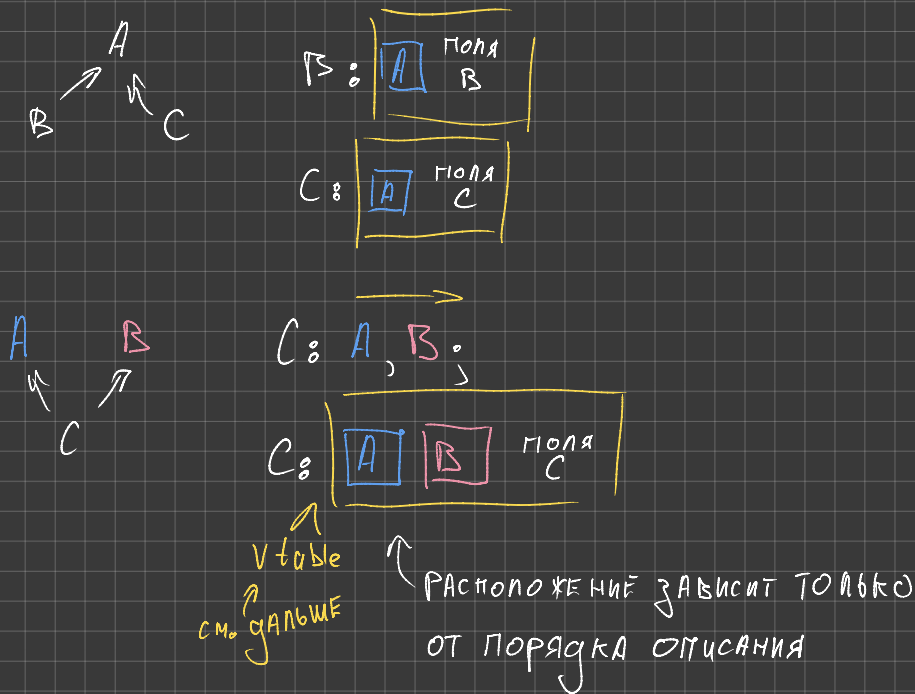
```
D: B;
```

```
E: C;
```

```
F: E, D;
```

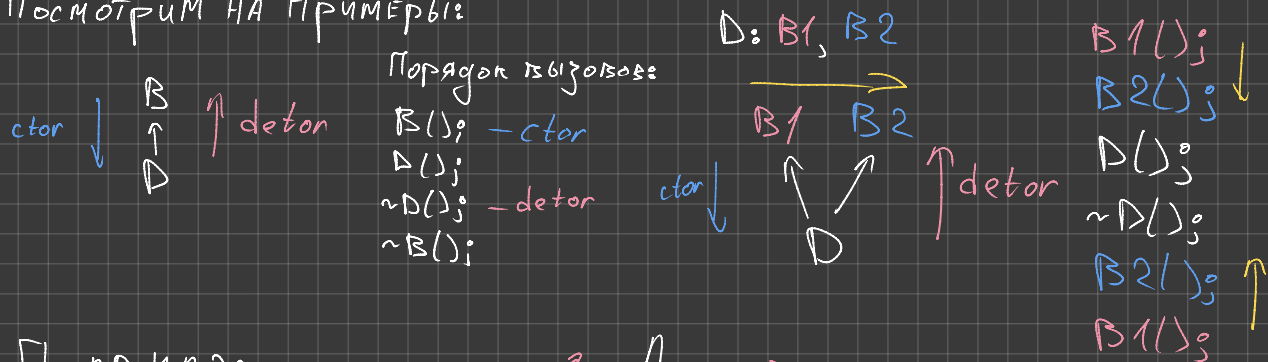


Укладка полей наследуемого класса в памяти



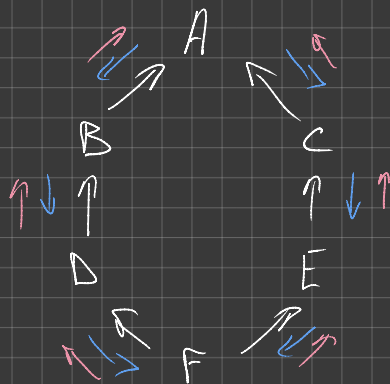
Конструкторы (ctor) и деструкторы (dctor)

Посмотрим на примеры:



Правило:

ctor ↓ ↑ ↓
dctor ↑ ↑ ↓



Почему так?

- Базовые классы ничего не знают о своих производных;
- Конструкторы производных классов могут использовать для инициализации настройки суперклассов;
- Разрушение базового класса подразумевает разрушение производного;
- Имеет смысл разрушить производный класс до того, как будет уничтожен весь объект